

Parallelizing Congruence Closure

Zachary Kent*, Amar Shah*
Carnegie Mellon University, Pittsburgh, PA, USA
{zkent, amarshah}@cmu.edu

Abstract—Given a set of ground equalities over terms with uninterpreted function symbols, congruence closure computes all ground equalities entailed by reflexivity, symmetry, transitivity, and congruence. While congruence closure is a performance-critical component of satisfiability modulo theories (SMT) solvers, automatic theorem provers, and equality saturation engines, all existing implementations are sequential. In this work, we provide the first parallel algorithm for congruence closure.

We build on two ideas from the parallel algorithms literature: a union-find data structure that supports concurrent union operations and a parallel semisort (group-by) algorithm that allows us to discover new congruences in parallel.

We present two algorithms. The first replaces the classic sequential worklist with a bulk-synchronous closure loop: it processes all pending unions in parallel, then detects new congruences, and repeats until a fixpoint. The second removes the global barrier between these phases, detecting congruences concurrently with union processing and feeding the resulting unions back into a shared worklist. We evaluate both these algorithms against a sequential baseline on a number of synthetic and real-world equality saturation benchmarks. Both of our algorithms can be extended to support backtracking, proof production, and dynamic term insertion. These algorithms are compatible with modern SMT solver and equality saturation architectures.

[1]

I. INTRODUCTION

II. BACKGROUND

III. ALGORITHM

IV. IMPLEMENTATION

V. EVALUATION

VI. DISCUSSION

REFERENCES

- [1] D. Alistarh, A. Fedorov, and N. Koval, “In search of the fastest concurrent union-find algorithm,” in *23rd International Conference on Principles of Distributed Systems, OPODIS 2019, Neuchâtel, Switzerland, December 17-19, 2019*, ser. LIPIcs, P. Felber, R. Friedman, S. Gilbert, and A. Miller, Eds. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, pp. 15:1–15:16. [Online]. Available: <https://doi.org/10.4230/LIPIcs.OPODIS.2019.15>

*Both authors contributed equally to this research.